

IoT-Based Weed Detection & Removal Robot

Phase 5 — Team Task Assignment & Working File Guide

Vision Integration: Camera Streaming, YOLOv8 Weed Detection & Automated ESP32 Actuation

Project	IoT-Based Weed Detection and Removal System
Current Status	Phase 4 (Movement, Cutter Automation, Sessions Dashboard) — COMPLETE
Next Phase	Phase 5 — Smart Detection (AI Vision Integration)
Team Size	4 Members
Repository	IoT-Based-Weed-Detection-Removal
Document Purpose	Assign detailed Phase 5 tasks to each member and specify the exact files/folders each person owns and should work inside.

This document consolidates the completed Phase 4 baseline with the Phase 5 roadmap (AI-based weed detection using YOLOv8, live camera streaming from a LILYGO T-SIMCAM module, and automated ESP32 actuation) into a single execution plan. Each team member's tasks are listed against the exact repository folders/files they are responsible for, so work can proceed in parallel without overlap.

1. Project Status Recap — Phase 4 (Completed)

Phase 4 delivered full movement and cutter automation with a live dashboard. The following is now in place and stable in the repository:

- ESP32 firmware (**esp32_phase4_v1_3.ino**) with PWM motor control via two L298N drivers, RS775 cutter relay control, and HC-SR04 obstacle avoidance (reverse → turn → retry).
- REST API contract: **/move**, **/stop**, **/cutter**, **/status**, plus Manual/Autonomous mode switching.
- Backend (Node.js/Express) fully modularized into **controllers/**, **routes/**, and **services/**, with session grouping (**GET /api/sessions**) and JSON-file persistence (**logs.json**, **sessions.json**, **settings.json**).
- Frontend dashboard with Robot Control, Field Map, Analytics, Alerts, and Settings pages, plus the new "Previous 5 Sessions" card grid wired to real backend data.
- Documentation set: API docs, wiring diagrams, Phase 4 Master Plan, component tracker, and STL chassis files.

Known follow-ups carried into Phase 5:

- **node_modules** was excluded from the Phase 4 handoff zip — run `npm install` inside **software/backend/** before starting the server.
- Session-gap threshold (**SESSION_GAP_MINUTES** in `server.js`) may need tuning once real camera-triggered events start arriving alongside simulated ones.

2. Phase 5 Overview — Smart Detection (AI Vision)

Phase 5 adds a vision pipeline on top of the working Phase 4 control system. A LILYGO T-SIMCAM (ESP32-S3 + OV5640) streams live video over Wi-Fi to a laptop (HP Victus, RTX 3050) running a YOLOv8 model. When a weed is detected with sufficient confidence, the detection script calls the existing backend API to trigger the cutter — no changes to the Phase 4 movement/cutter firmware are required, only a new inbound trigger path.

Phase 5 Roadmap (from project execution guide):

- Install and bring up the LILYGO T-SIMCAM (ESP32-S3 + OV5640) camera streaming module.
- Create and label a weed/crop image dataset for the target crop.
- Train a YOLOv8 model (transfer learning from COCO weights) to classify weed vs. crop.
- Integrate real-time detection — camera feed → YOLOv8 inference → confidence-gated trigger.
- Wire the detection trigger into ESP32 command automation (reuse the existing cutter API).
- Field-test the full pipeline end-to-end and log accuracy/response-time results.

Success criteria for Phase 5:

- YOLOv8 achieves $\geq 85\%$ precision/recall on the validation set for weed vs. crop.
- End-to-end latency from camera frame to cutter activation stays under ~ 1.5 seconds.
- Live camera feed and detection overlay are visible on the dashboard.
- Full demo: camera detects a weed → backend receives trigger → cutter fires → event is logged and shown in Activity Log/Sessions.

3. Updated Repository Structure for Phase 5

New folders/files to be added on top of the existing Phase 4 structure are marked **[NEW]**. Everything else already exists from Phase 4 and should not be restructured.

```
IoT-Based-Weed-Detection-Removal/
|-- docs/
|   |-- project-plan.md
|   |-- api-documentation.md
|   |-- phase5-test-report.md           [NEW - Member 4]
|   |-- dataset-collection-guide.md     [NEW - Member 4]
|   `-- Resources/
|       |-- Phase4_Plan.pdf
|       `-- Phase5_Plan.pdf             [NEW - Member 4]
|-- hardware/
|   `-- ESP-32/
|       |-- sketch_apr12a_v3/esp32_phase4_v1_3/ (existing, stable)
|       `-- sketch_phase5_vision/           [NEW - Member 3]
|           |-- sketch_phase5_vision.ino
|-- software/
|   |-- ai-vision/                       [NEW - Member 3 + Member 1 support]
|   |   |-- dataset/
|   |   |   |-- raw/
|   |   |   |-- labeled/
|   |   |   `-- annotations.yaml
|   |   |-- models/
|   |   |   `-- yolov8_weed.pt
|   |   `-- scripts/
|   |       |-- train.py
|   |       |-- detect_stream.py
|   |       `-- requirements.txt
|   |-- backend/
|   |   |-- controllers/
|   |   |   `-- detectionController.js     [NEW - Member 1]
|   |   |-- routes/
|   |   |   `-- detection.js              [NEW - Member 1]
|   |   `-- services/
|   |       `-- esp32Service.js           (extend - Member 1)
|   |-- database/
|   |   `-- schema.md                    (extend - Member 1)
|   `-- frontend/
|       |-- pages/
|       |   `-- camera-feed.html         [NEW - Member 2]
|       |-- js/
|       |   `-- camera-feed.js           [NEW - Member 2]
|       `-- css/
|           `-- camera-feed.css          [NEW - Member 2]
```

4. New API Contract for Phase 5

These new endpoints extend the existing Phase 4 contract. No existing endpoint changes.

Method / Endpoint	Purpose	Owner
POST /api/detection/weed	Called by the YOLOv8 detect_stream.py script when a weed is identified above the confidence threshold. Triggers the existing cutter logic and logs the event.	sMember 1

GET /api/detection/status	Returns latest detection state (last detection time, confidence, model version) for the dashboard camera-feed page.	Member 1
GET /api/detection/stream	Proxies / exposes the live camera feed URL to the frontend.	Member 1 + Member 2

5. Detailed Task Assignment by Team Member

Member 1 — Backend Developer

Objective: Build the API bridge between the YOLOv8 detection script and the existing ESP32 cutter/movement control layer, and expose detection status/history to the frontend.

Folders/files to work in:

- **software/backend/routes/detection.js** — new route file for detection endpoints.
- **software/backend/controllers/detectionController.js** — new controller handling incoming weed-detection events.
- **software/backend/services/esp32Service.js** — extend to forward detection-triggered cutter commands to the ESP32.
- **software/backend/data/logs.json** — detection events should be appended using the existing log schema (add a source: "ai-vision" field).
- **software/database/schema.md** — document the new detection-event schema.

Tasks:

- Implement **POST /api/detection/weed**: validate payload (confidence, timestamp, frame id), then reuse the existing cutter-trigger logic from Phase 4 (do not duplicate it — call the same internal function used by **/api/cutter**).
- Implement **GET /api/detection/status** returning last detection time, confidence score, and current model version.
- Add a debounce/cooldown (e.g. 3–5 seconds) so rapid repeated detections don't fire the cutter continuously.
- Extend **groupIntoSessions()** in **server.js** so AI-triggered detections are counted alongside manual/simulated ones in the session cards.
- Write a short Postman collection or curl examples for the two new endpoints and hand off to Member 4 for the API documentation.

Member 2 — Frontend Developer

Objective: Give the dashboard a live camera view with detection overlay so the weed-detection pipeline is visible and demoable in real time.

Folders/files to work in:

- **software/frontend/pages/camera-feed.html** — new page for the live stream + detection overlay.
- **software/frontend/js/camera-feed.js** — new script to poll **/api/detection/status** and render the video/overlay.
- **software/frontend/css/camera-feed.css** — styling consistent with existing pages (reuse variables from **style.css**).
- **software/frontend/index.html** — add a navigation link / card to the new Camera Feed page.
- **software/frontend/js/api.js** — add helper functions for the two new detection endpoints.

Tasks:

- Build the Camera Feed page matching the existing visual style (same color variables/shadows/radii as Robot Control and Analytics pages).
- Render the MJPEG/HTTP stream from the T-SIMCAM (or the proxied URL from Member 1's **/api/detection/stream**).

- Show a live badge ("Weed Detected" / "Clear") driven by **/api/detection/status**, polling every 2–3 seconds like the rest of the dashboard.
- Add the last-detection confidence score and timestamp to the existing Activity Log panel styling.
- Cross-check with Member 1 on the exact JSON shape of **/api/detection/status** before wiring up rendering.

Member 3 — ESP32 / Vision Hardware Developer (You)

Objective: Bring up the LILYGO T-SIMCAM streaming hardware and build the YOLOv8 detection pipeline that watches the stream and calls the backend when a weed is found.

Folders/files to work in:

- **hardware/ESP-32/sketch_phase5_vision/sketch_phase5_vision.ino** — new firmware for the LILYGO T-SIMCAM (camera init, Wi-Fi streaming server); the existing **sketch_apr12a_v3** Phase 4 firmware stays untouched and keeps running on the drive-motor ESP32.
- **software/ai-vision/dataset/** — raw and labeled weed/crop images plus **annotations.yaml**.
- **software/ai-vision/models/** — trained **yolov8_weed.pt** weights.
- **software/ai-vision/scripts/train.py** — training script (transfer learning from COCO-pretrained YOLOv8).
- **software/ai-vision/scripts/detect_stream.py** — real-time inference script: reads the camera stream, runs YOLOv8, and calls **POST /api/detection/weed** on a positive detection.
- **software/ai-vision/scripts/requirements.txt** — Python deps (ultralytics, opencv-python, requests, etc.).

Tasks:

- Flash and bring up the T-SIMCAM (ESP32-S3 + OV5640): verify Wi-Fi connection and a stable MJPEG/HTTP video stream.
- Collect the weed/crop image dataset from the test field/tray; organize into **dataset/raw/** then label (e.g. Labellmg) into **dataset/labeled/**.
- Train YOLOv8 in **train.py**: 80/10/10 split, ~100 epochs, batch size 16, Adam optimizer (lr 0.001), transfer learning from COCO weights.
- Build **detect_stream.py** to run inference at ~1 FPS on frames from the stream, apply a confidence threshold (>50%), and POST to the backend when a weed is confirmed.
- Confirm the two ESP32s coexist cleanly on the same Wi-Fi network (drive/cutter ESP32 from Phase 4 + camera ESP32-S3) without address or port conflicts.
- Add a hardware safety timer/emergency-stop check before any cutter trigger reaches the motor driver.

Member 4 — QA, Documentation & Integration

Objective: Own end-to-end testing of the new vision pipeline, keep documentation in sync, and coordinate integration between the three other members' work.

Folders/files to work in:

- **docs/*** — all project documentation (this is your primary folder, unchanged from Phase 4).
- **docs/Resources/Phase5_Plan.pdf** — new Phase 5 master plan document (mirrors the Phase4_Plan.pdf format already in the repo).
- **docs/dataset-collection-guide.md** — new guide describing how/where the weed/crop dataset was collected and labeled, for reproducibility.
- **docs/phase5-test-report.md** — new test log for detection accuracy, latency, and end-to-end demo runs.

- **docs/api-documentation.md** — extend with the two new detection endpoints from Member 1.
- **docs/Resources/component_tracker.xlsx** — add the T-SIMCAM and any new Phase 5 hardware line items.

Tasks:

- Write the wiring/setup diagram for the T-SIMCAM alongside the existing Phase 4 wiring diagrams.
- Build a test plan covering: detection accuracy (precision/recall on held-out images), end-to-end latency (frame → cutter activation), false-positive rate, and dual-ESP32 network stability.
- Run integration tests across all three other members' work once each piece lands — verify the full loop: camera → YOLOv8 → backend → cutter → dashboard log.
- Track bugs/issues in a shared tracker and confirm fixes before sign-off.
- Prepare the Phase 5 demo script and final report/PPT sections covering the vision pipeline.

6. Suggested Phase 5 Timeline

Week	Focus	Primary Owner(s)
Week 1	T-SIMCAM bring-up + stream verification; scaffold ai-vision/ folder and requirements.txt	Member 3
Week 2	Dataset collection + labeling; start detection.js / detectionController.js scaffold	Member 3, Member 1
Week 3	YOLOv8 training (train.py); backend detection endpoints functional with mock payload	Member 3, Member 1
Week 4	detect_stream.py live inference wired to backend; camera-feed.html/js first pass	Member 3, Member 1, Member 2
Week 5	Full pipeline integration (camera → YOLOv8 → backend → cutter → dashboard); QA test plan execution	All + Member 4 lead
Week 6	Field testing, bug fixing, docs/report finalization, demo rehearsal	All + Member 4 lead

7. Cross-Team Coordination Notes

- Member 1 and Member 3 must agree early on the exact JSON payload for **POST /api/detection/weed** (fields: confidence, timestamp, frame_id, label).
- Member 2 should not start the camera-feed UI until Member 1's **/api/detection/status** response shape is finalized, to avoid rework.
- Member 4 should sit in on the Week 4 integration checkpoint to start drafting the test plan against real (not mock) endpoints.
- All new Phase 5 code should branch off **develop** per the existing Git workflow (**backend-dev**, **frontend-dev**, **esp32-dev**, plus a new **vision-dev** branch for the ai-vision/ folder).